Rideshare Dynamic Pricing System Project Report

End-to-End Machine Learning, Simulation, Explainability, and Deployment Pipeline

Taksh Girdhar

Overview

This project develops an end-to-end dynamic pricing system for rideshare scenarios. The system begins with a structured feature engineering pipeline, compares multiple machine learning models, evaluates model diagnostics, simulates pricing policies under a probabilistic acceptance assumption, adds explainability through SHAP and local feature contributions, and deploys the final model through a FastAPI service with Docker and Kubernetes-ready infrastructure. The selected production model is Ridge Regression using the v1 price target configuration, chosen for its strong predictive performance, stability, interpretability, and simpler deployment requirements.

Contents

1 Introduction

Dynamic pricing is the process of adjusting prices based on contextual, operational, and demand-related factors. In a rideshare setting, pricing may depend on demand pressure, supply availability, trip duration, vehicle type, location, time of booking, and rider characteristics.

The objective of this project is to build a production-oriented dynamic pricing system rather than a standalone notebook model. The project includes:

- strict data validation and feature engineering,
- model benchmarking and statistical diagnostics,
- model versioning and artifact management,
- counterfactual pricing policy simulation,
- model explainability,
- FastAPI model serving,
- Docker containerization,
- Kubernetes-ready deployment configuration.

Placeholder: Add a high-level architecture diagram here.

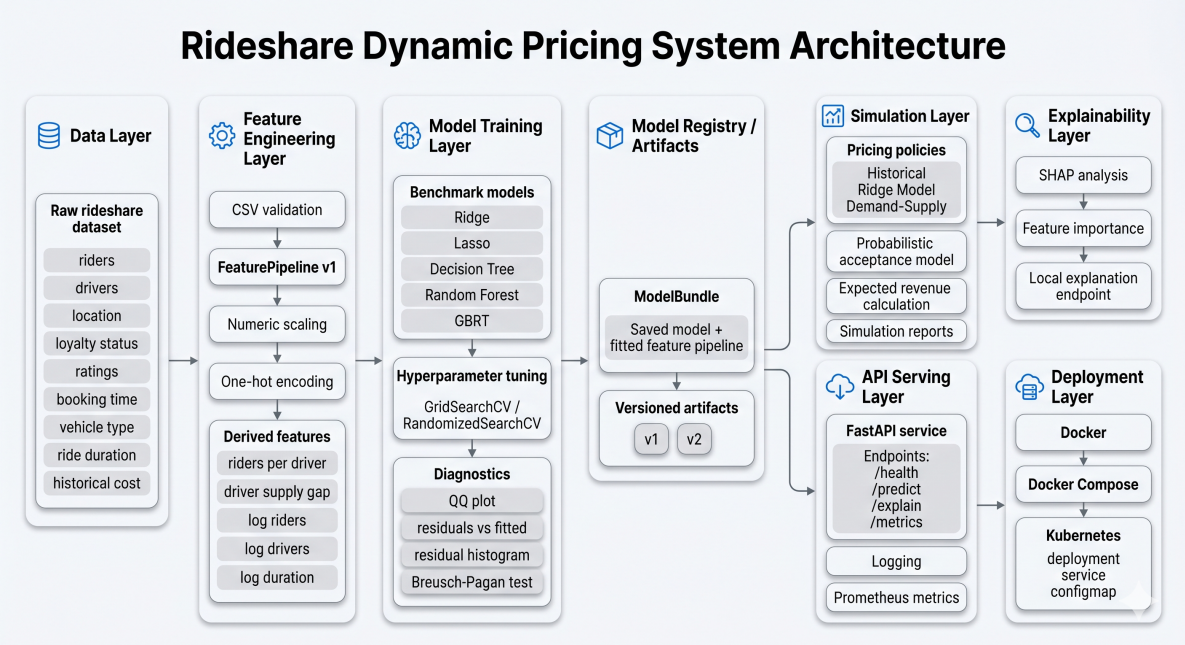


Figure 1: High-level architecture of the dynamic pricing system.

2 Dataset and Problem Definition

2.1 Dataset Overview

The dataset contains ride-level contextual information and historical ride prices. The prediction target for the main modeling task is:

$$y = \texttt{Historical\_Cost\_of\_Ride}$$

Each row represents a ride context with information about demand, supply, customer history, time of booking, vehicle type, expected duration, and historical cost.

2.2 Raw Dataset Columns

Table 1: Dataset schema used in the project.

Column	Description
Number_of_Riders	Number of riders/request demand in the context.
Number_of_Drivers	Number of available drivers/supply in the context.
Location_Category	Location type such as Urban, Suburban, or Rural.
Customer_Loyalty_Status	Customer loyalty category.
Number_of_Past_Rides	Customer ride history count.
Average_Ratings	Customer average rating.
Time_of_Booking	Time bucket for the booking.
Vehicle_Type	Vehicle category such as Economy or Premium.
Expected_Ride_Duration	Estimated trip duration.
Historical_Cost_of_Ride	Historical price of the ride; used as target.

3 Repository and System Design

The project was structured as a production-style Python package rather than a notebook-only analysis. The main package is stored under:

```
src/dynamic_pricing/
```

The important folders include:

```
src/dynamic_pricing/
  api/                  FastAPI app, routes, dependencies, middleware
  features/             Data loading and feature pipeline
  models/               Training, evaluation, registry, model bundle
  simulation/           Pricing policies, acceptance model, simulator
  explainability/       SHAP analysis pipeline
  schemas/              Pydantic request/response schemas
  config/               Settings and logging utilities

artifacts/
  model/v1/             Production model bundle
```

```
  model/v2/             Log-target model bundle

reports/
  figures/v1/           v1 diagnostic figures
  figures/v2/           v2 diagnostic figures
  figures/shap/         SHAP plots
  simulation_results.md Simulation summary output
```

4 Feature Engineering Pipeline

4.1 Design Goal

The feature pipeline was designed as the single source of truth for all downstream stages:

- model training,
- simulation,
- API inference,
- explainability.

This avoids training-serving skew because the same fitted feature pipeline is saved with the model artifact.

4.2 Numeric Features

The raw numeric features are:

- `Number_of_Riders`
- `Number_of_Drivers`
- `Number_of_Past_Rides`
- `Average_Ratings`
- `Expected_Ride_Duration`

These are standardized using `StandardScaler`.

4.3 Derived Features

Several derived features were created to capture demand pressure, supply imbalance, and long-tail effects:

$$\texttt{riders\_per\_driver} = \frac{\texttt{Number\_of\_Riders}}{\max(\texttt{Number\_of\_Drivers}, 1)} \tag{1}$$

$$\texttt{driver\_supply\_gap} = \texttt{Number\_of\_Drivers} - \texttt{Number\_of\_Riders} \tag{2}$$

$$\texttt{log\_riders} = \log(1 + \texttt{Number\_of\_Riders}) \tag{3}$$

$$\texttt{log\_drivers} = \log(1 + \texttt{Number\_of\_Drivers}) \tag{4}$$

$$\texttt{log\_duration} = \log(1 + \texttt{Expected\_Ride\_Duration}) \tag{5}$$

4.4 Categorical Features

Categorical variables were encoded using one-hot encoding with unknown categories ignored at inference time:

- `Location_Category`
- `Customer_Loyalty_Status`
- `Time_of_Booking`
- `Vehicle_Type`

4.5 Feature Pipeline Artifact

The fitted feature pipeline is saved as part of the model bundle. This ensures that the exact same transformations used during training are reused during inference.

`artifacts/model/v1/model_bundle.joblib`

Table 2: Feature engineering summary.

Feature Group	**Features**	**Transformation**
Raw numeric	`Number_of_Riders, Number_of_Drivers, Number_of_Past_Rides, Average_Ratings, Expected_Ride_Duration`	StandardScaler
Derived numeric	`riders_per_driver, driver_supply_gap, log_riders, log_drivers, log_duration`	StandardScaler
Categorical	`Location_Category,Customer_Loyalty_Status, Time_of_Booking, Vehicle_Type`	OneHotEncoder

5 Model Benchmarking

5.1 Models Evaluated

The following models were benchmarked:

- Ridge Regression
- Lasso Regression
- Decision Tree Regressor
- Random Forest Regressor
- Gradient Boosting Regressor

Hyperparameter tuning was performed using `GridSearchCV` and `RandomizedSearchCV`, depending on the model and search space size.

5.2 Evaluation Metrics

The models were evaluated using:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2
- Breusch–Pagan test for heteroskedasticity
- Residual diagnostics

5.3 v1 Model Results: Price Target

The first model version, v1, directly predicted historical ride price.

Table 3: v1 benchmark results.

Model	**MAE**	**RMSE**	R^2	**BP p-value**
Ridge	52.491	67.435	0.875	1.397×10^{-10}
Lasso	52.092	67.564	0.875	2.833×10^{-10}
GBRT	54.107	70.625	0.863	8.060×10^{-9}
Random Forest	54.404	72.665	0.855	5.416×10^{-9}
Decision Tree	56.552	74.055	0.850	1.220×10^{-10}

5.4 v2 Model Results: Log-Transformed Target

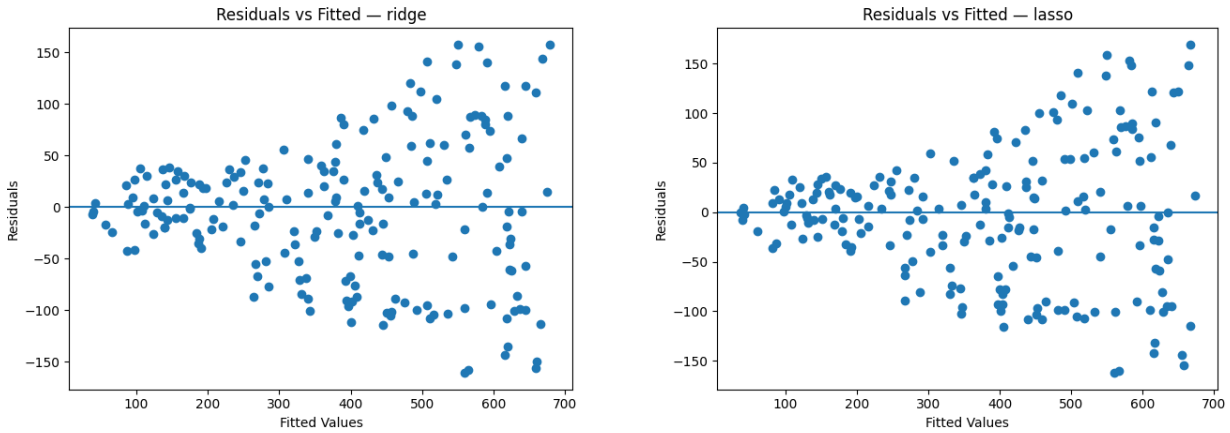
The v2 experiment trained models on a log-transformed target and then back-transformed predictions for evaluation.

Table 4: v2 benchmark results.

Model	**MAE**	**RMSE**	R^2	**BP p-value**
Lasso	52.337	67.201	0.876	5.349×10^{-11}
Ridge	52.656	68.498	0.871	1.079×10^{-9}
GBRT	53.257	69.650	0.867	4.373×10^{-9}
Random Forest	54.913	73.057	0.854	6.062×10^{-9}
Decision Tree	57.051	75.049	0.846	7.567×10^{-11}

5.5 Diagnostic Plots

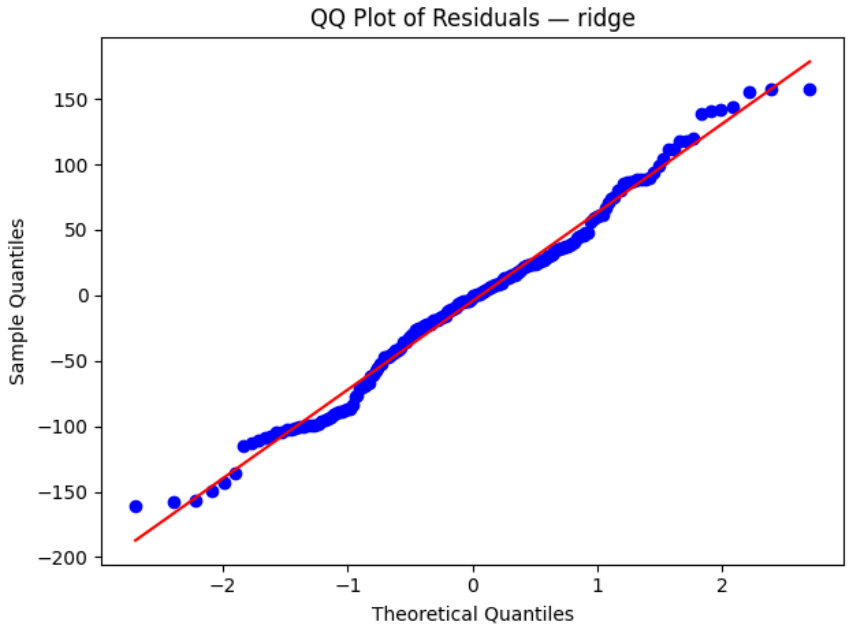
Residual plots showed a funnel pattern, indicating heteroskedasticity: error variance increased with fitted price.



(a) Ridge v1

(b) Lasso v1

Figure 2: Residual-vs-fitted diagnostics for v1 linear models.



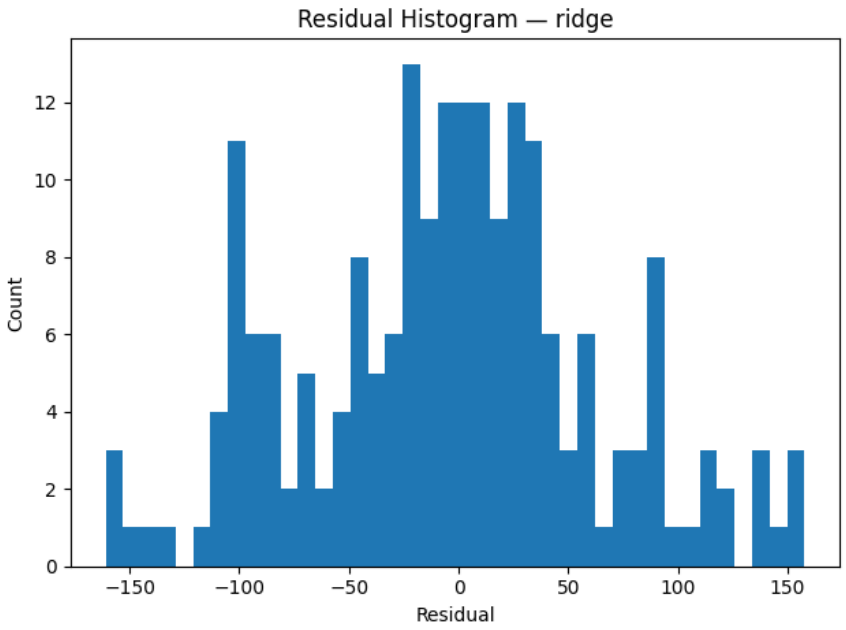


Figure 3: Residual histogram for Ridge v1.

6 Model Selection Decision

Although Lasso v2 achieved a slightly lower RMSE than Ridge v1, the improvement was small. Ridge v1 was selected as the production candidate because it provides:

- strong predictive performance,
- stable coefficient shrinkage under correlated engineered features,
- simpler interpretation,
- simpler deployment because no inverse log-transform is required.

Table 5: v1 vs v2 comparison.

Aspect	**v1**	**v2**
Best model	Ridge	Lasso
Best RMSE	67.435	67.201
Best R^2	0.875	0.876
Target	Price	Log(price)
Heteroskedasticity	Strong	Slightly reduced
Deployment complexity	Lower	Higher
Production decision	Selected	Alternative

The final production model is:

Ridge Regression v1

7 Counterfactual Pricing Simulation

7.1 Purpose

After model benchmarking, the project was extended from prediction to decision-making. A counterfactual simulation framework was implemented to compare different pricing policies under a controlled acceptance model.

7.2 Policies Simulated

The following policies were evaluated:

- **Historical Policy**: returns the original historical price.
- **Ridge Model Policy**: uses the selected Ridge v1 model to generate predicted prices.
- **Demand-Supply Policy**: applies a rule-based multiplier based on rider-driver imbalance.

7.3 Acceptance Probability Model

Because the dataset does not include true ride acceptance/rejection labels, a probabilistic acceptance model was introduced.

$$P(\text{accept}) = \frac{1}{1 + e^{k\Delta}}$$

where:

$$\Delta = \frac{\text{proposed price} - \text{base price}}{\text{base price}}$$

The intuition is that customer acceptance decreases as the proposed price increases relative to the historical baseline price.

7.4 Expected Revenue

Expected revenue is defined as:

$$\text{Expected Revenue} = \text{Proposed Price} \times P(\text{accept})$$

This captures the tradeoff between higher prices and lower acceptance probability.

7.5 Simulation Outputs

Table 6: Simulation policy comparison.

Policy	Avg. Price	Avg. Acceptance	Total Expected Revenue	Price Std.
Historical	372.503	0.5	186251	187.159
Ridge Model	373.383	0.475642	177739	175.03
Demand-Supply	612.915	0.647995	70140.7	330.926

7.6 Simulation Limitations

The simulation is not causal. The acceptance function is a heuristic because the dataset does not contain:

- true acceptance/rejection labels,
- demand at different price levels,
- customer willingness-to-pay,
- elasticity estimates.

Therefore, the simulation should be interpreted as a controlled policy comparison framework rather than a real-world revenue forecast.

8 Explainability

8.1 Global Explainability

SHAP was used to understand which features most influence predicted ride prices. The SHAP analysis was performed after feature transformation, meaning explanations reflect the exact feature space seen by the model.

Figure 4: Global feature importance using mean absolute SHAP values.

Figure 5: SHAP beeswarm plot showing feature impact magnitude and direction.

Table 7: Top SHAP feature importance placeholder.

Feature	Mean Absolute SHAP
Expected_Ride_Duration	162.44404286171962
Vehicle_Type_Economy	10.542881412319131
Vehicle_Type_Premium	10.542881412318586
log_riders	7.2135774496876675
log_duration	6.38423321968597
Number_of_Riders	5.603018026989024
Number_of_Drivers	4.095298087670915
riders_per_driver	2.8494357770777943
log_drivers	2.748225017126227
driver_supply_gap	2.4907892505842324

8.2 Local API Explainability

In addition to offline SHAP plots, the deployed API includes an `/explain` endpoint. For linear models such as Ridge, the local contribution of a transformed feature is computed as:

$$\text{Contribution}_j = x_j \beta_j$$

where x_j is the transformed feature value and β_j is the learned model coefficient.

This endpoint returns the top feature contributions for a single prediction.

9 API Deployment

9.1 FastAPI Service

The selected model was served through a FastAPI application. The API supports:

- `GET /health`: service health check,
- `POST /predict`: predicted ride price,
- `POST /explain`: prediction with local feature contributions,
- `GET /metrics`: Prometheus-style metrics endpoint.

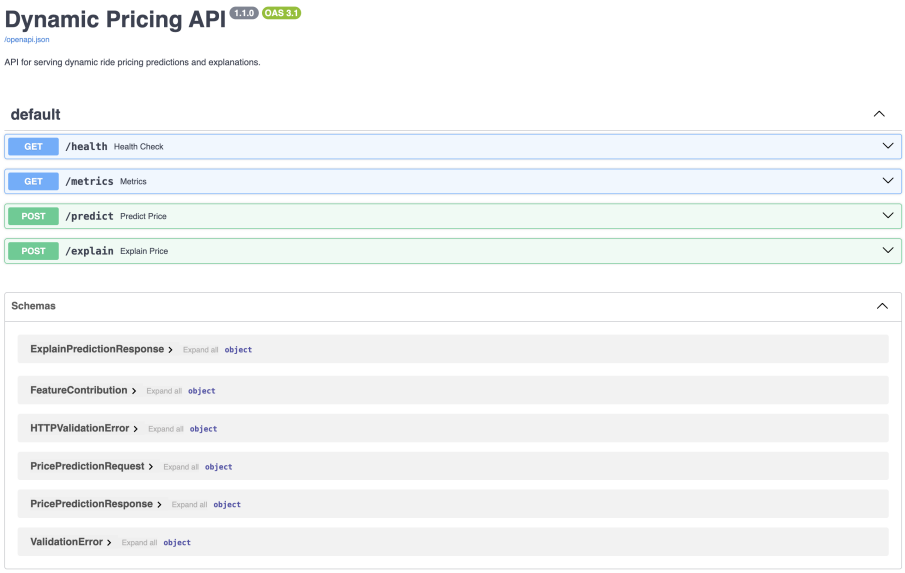


Figure 6: FastAPI documentation page.

9.2 API Request Example

```
{
  "Number_of_Riders": 60,
  "Number_of_Drivers": 25,
  "Location_Category": "Urban",
  "Customer_Loyalty_Status": "Gold",
  "Number_of_Past_Rides": 10,
  "Average_Ratings": 4.2,
  "Time_of_Booking": "Evening",
  "Vehicle_Type": "Economy",
  "Expected_Ride_Duration": 40,
  "Historical_Cost_of_Ride": 200.0
}
```

Example API response for `/predict`.

```
{
  "predicted_price": 132.66,
  "model_name": "ridge",
  "model_version": "v1",
  "feature_version": "v1"
}
```

Example API response for `/explain`.

```
{
  "predicted_price": 132.66,
  "model_name": "ridge",
  "model_version": "v1",
```

```
"feature_version": "v1",
"top_contributions": [
  {
    "feature": "Expected_Ride_Duration",
    "value": -1.2066255661337626,
    "contribution": -215.11850453134377
  },
  {
    "feature": "Vehicle_Type_Economy",
    "value": 1,
    "contribution": -21.097577468020287
  },
```


9.3 Logging and Monitoring

The API includes request logging middleware. Each request records:

- HTTP method,
- endpoint path,
- response status,
- request duration.

The monitoring layer exposes Prometheus-compatible metrics, including:

- total request count by endpoint,
- request latency histograms.


```
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="0.005",method="POST"} 0.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="0.01",method="POST"} 1.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="0.025",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="0.05",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="0.075",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="0.1",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="0.25",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="0.5",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="0.75",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="1.0",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="2.5",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="5.0",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="7.5",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="10.0",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/predict",le="+Inf",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_count{endpoint="/predict",method="POST"} 2.0
dynamic_pricing_http_request_duration_seconds_sum{endpoint="/predict",method="POST"} 0.0316319465637207
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/explain",le="0.005",method="POST"} 0.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/explain",le="0.01",method="POST"} 0.0
dynamic_pricing_http_request_duration_seconds_bucket{endpoint="/explain",le="0.025",method="POST"} 1.0
```


Figure 7: Prometheus-compatible metrics endpoint.

10 Docker and Kubernetes Deployment

10.1 Docker

The project includes a Dockerfile that packages the application, dependencies, source code, and model artifacts into a containerized FastAPI service.

The service can be built and run with:

```
docker build -t dynamic-pricing-api:v1 .
docker run --rm -p 8000:8000 dynamic-pricing-api:v1
```

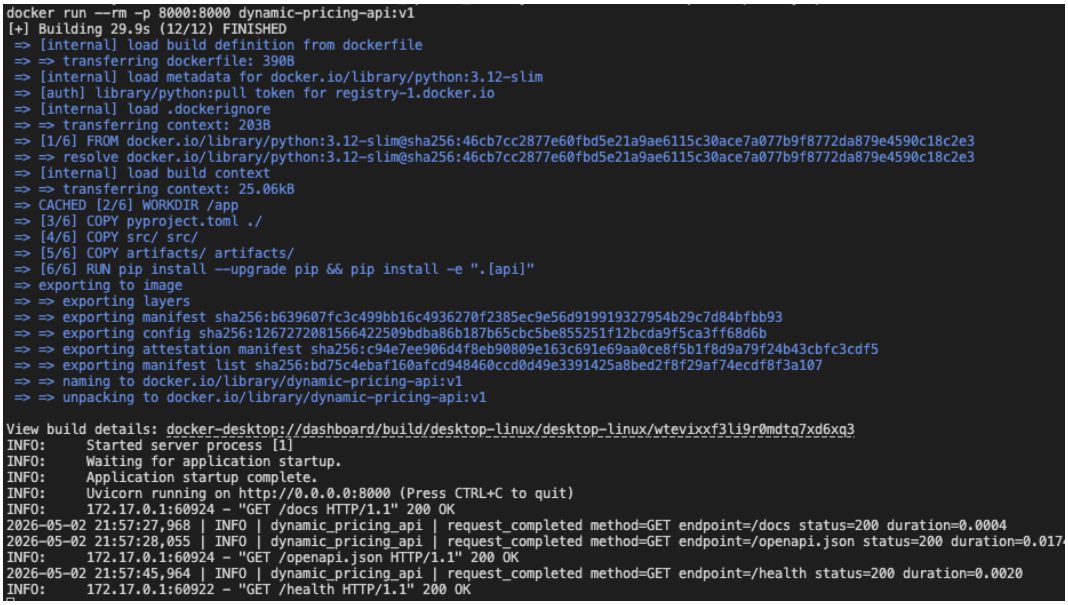


Figure 8: Dockerized FastAPI service running locally.

10.2 Kubernetes

Kubernetes deployment manifests were prepared under:

`deploy/k8s/base/`

The Kubernetes setup includes:

- `deployment.yaml`: API deployment with replicas,
- `service.yaml`: internal service routing,
- `configmap.yaml`: environment-level configuration.

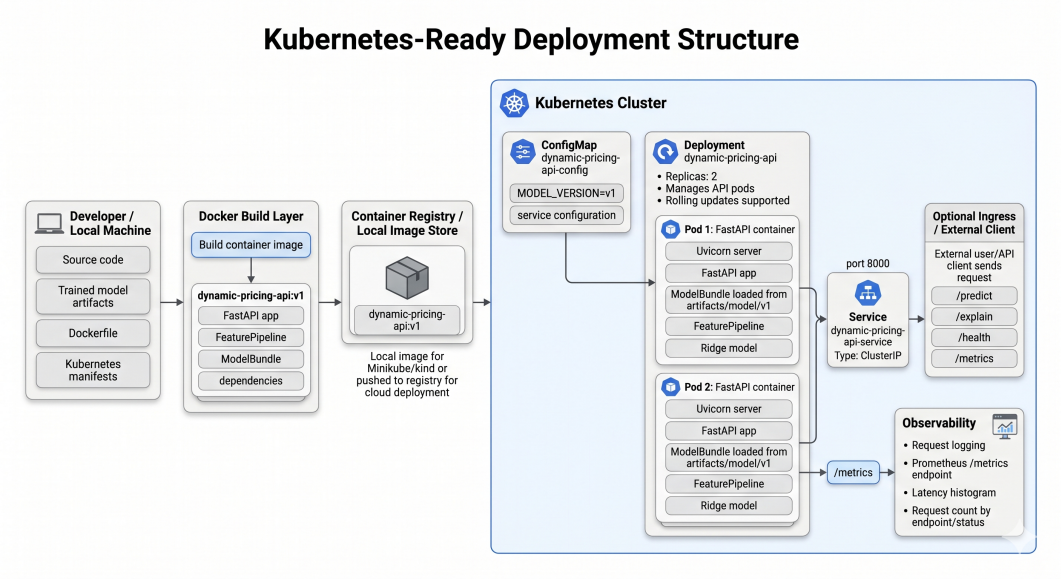


Figure 9: Kubernetes-ready deployment structure.

11 Evaluation and Improvement Metrics

In addition to standard regression metrics, the project evaluates improvement across three dimensions: predictive performance, pricing-policy behavior, and explainability. This provides a more complete view of the system as a deployable pricing decision framework rather than only a prediction model.

11.1 Model Improvement over Naive Baseline

A naive baseline was defined as a model that always predicts the mean historical ride price from the training split. Model improvement is measured relative to this baseline using MAE reduction, RMSE reduction, and R^2 lift.

$$\text{RMSE Reduction} = \frac{\text{RMSE}_{baseline} - \text{RMSE}_{model}}{\text{RMSE}_{baseline}} \times 100 \tag{6}$$

$$\text{MAE Reduction} = \frac{\text{MAE}_{baseline} - \text{MAE}_{model}}{\text{MAE}_{baseline}} \times 100 \tag{7}$$

$$R^2 \text{ Lift} = R^2_{model} - R^2_{baseline} \tag{8}$$

Table 8: Model improvement over naive mean-price baseline.

Version	Model	MAE	RMSE	R^2	MAE Reduction	RMSE Reduction	R^2 Lift
Baseline	Mean Price	160.964	191.153	-0.002	0.000%	0.000%	0.000
v1	Ridge	52.491	67.435	0.875	67.390%	64.722%	0.877
v2	Lasso (log target)	52.337	67.201	0.876	67.485%	64.845%	0.878

The v1 Ridge model reduced RMSE by approximately 64.72% compared with the naive baseline. The v2 Lasso model achieved a marginally lower RMSE, but the improvement over v1 was only 0.235 RMSE points.

This supports the earlier decision to retain Ridge v1 as the production candidate because the v2 gain was too small to justify the added inverse-transform deployment complexity.

Table 9: v2 improvement relative to v1.

Metric	v2 Delta vs v1
RMSE Delta	-0.235
MAE Delta	-0.154
R^2 Delta	0.001

11.2 Pricing Policy Improvement Metrics

The pricing simulation was evaluated using policy-level metrics that capture not only expected revenue but also acceptance probability, surge behavior, and customer impact.

Table 10: Pricing policy improvement metrics from counterfactual simulation.

Policy	Avg. Price	Avg. Acceptance	Expected Revenue	Revenue Lift	Surge >25%	Surge >50%	Customer Pain
Historical	372.503	0.500	186251.31	0.000%	0.000%	0.000%	0.000%
Ridge Model	373.383	0.476	177738.96	-4.570%	13.400%	0.900%	9.076%
Demand-Supply	612.915	0.136	70140.45	-62.341%	81.700%	60.400%	64.800%

The Ridge Model policy produces prices close to the historical baseline, with an average price increase of only 0.236% relative to historical pricing. However, the probabilistic acceptance model slightly reduces expected revenue because even small price changes can reduce acceptance probability. The Demand-Supply policy generates much higher prices and substantially higher surge frequency, but this sharply reduces simulated acceptance and expected revenue.

These results show that aggressive rule-based surge pricing can appear attractive from a price perspective but may underperform once acceptance probability is included.

11.3 Explainability Improvement Metrics

Explainability was evaluated using SHAP concentration metrics. These metrics measure how much of the model's total feature importance is explained by the most influential features.

Table 11: SHAP-based explainability concentration metrics.

Metric	Value
Top Feature	`Expected_Ride_Duration`
Top Feature Mean Absolute SHAP	162.444
Top-5 Feature Coverage	88.315%
Top-1 Feature Share	72.776%
Number of Features Explained	21

The SHAP metrics show that the model is highly concentrated around a small number of important features. In particular, `Expected_Ride_Duration` alone accounts for approximately 72.78% of total mean absolute SHAP importance, while the top five features account for approximately 88.32%. This indicates that the model's pricing behavior is dominated by a small, interpretable set of drivers.

11.4 Summary of Improvement Metrics

Overall, the improvement metrics show that:

- the selected Ridge v1 model substantially improves over a naive mean-price baseline,
- the log-target v2 experiment provides only marginal additional predictive improvement,
- the Ridge Model policy is more stable than the Demand-Supply policy under the simulated acceptance model,
- aggressive demand-supply pricing leads to high surge frequency and lower expected revenue,
- SHAP importance is highly concentrated, making the model relatively interpretable.

12 Key Results and Discussion

12.1 Main Findings

The main findings are:

1. Engineered features made regularized linear models highly competitive.
2. Ridge and Lasso outperformed tree-based models, suggesting the dominant pricing structure was captured by engineered linear signals.
3. Residual diagnostics showed heteroskedasticity, especially at higher predicted prices.
4. Log-transforming the target produced only marginal improvement.
5. Ridge v1 was selected for production because it offered strong performance and simpler deployment.
6. Counterfactual simulation extended the system from prediction to pricing policy evaluation.
7. Explainability and API observability made the system more transparent and production-ready.

12.2 Limitations

The main limitations are:

- The dataset does not include true ride acceptance or rejection outcomes.
- The simulation acceptance model is heuristic, not learned.
- The pricing model predicts historical cost rather than directly optimizing profit.
- Long-term pricing effects, customer retention, competition, and driver behavior are not modeled.
- SHAP explanations are model explanations, not causal explanations.

13 Future Work

Future extensions include:

- learning acceptance probability from real conversion data,
- estimating true price elasticity,
- adding prediction intervals or quantile regression,
- implementing constrained price optimization,
- adding fairness constraints across locations or customer segments,
- integrating Prometheus and Grafana dashboards,
- deploying to a live Kubernetes cluster,
- adding CI/CD for testing, Docker builds, and deployment.

14 Conclusion

This project developed a full dynamic pricing system from data preprocessing to deployment. The final system includes a validated feature pipeline, benchmarked machine learning models, model versioning, counterfactual policy simulation, explainability, FastAPI deployment, Docker containerization, and Kubernetes-ready infrastructure.

The selected production model is Ridge Regression v1. This choice balances predictive performance, interpretability, stability, and deployment simplicity. The project demonstrates not only machine learning modeling, but also the broader engineering, diagnostic, and governance practices required to build a deployable ML pricing system.